Horizon: confidential admission over two signed commitments

Relation, conditional proofs and experimental limitations

Technical dossier of the ZK-agentiK repository

17 October 2026 Experimental version 1

Abstract

Horizon verifies an inequality between an amount and a per-case limit, attested by two distinct roles, before recording a synthetic authorization. It uses Pedersen over Ristretto, Bulletproofs and Ed25519 signatures. This text provides conditional derivations, separates confidentiality, authenticity and recovery, and supplies executable arithmetic checks. It is neither a machine-checked formal proof, a cryptographic audit, a new primitive nor banking qualification. Octave tests do not prove cryptographic security. The profile is not post-quantum.

Contents

1 Purpose, scope and correspondence with the code **2**

2 Parameters and issuance contract **2**

3 Relation and conditional derivations **2**

3.1 Commitment: confidentiality and binding 2

3.2 The statement actually proved 3

4 Composition with signatures and challenge **4**

5 Replay prevention and transaction boundary **5**

6 Leakage and business counterexamples **5**

7 CPU, GPU and Vega measurements, and quantum assumptions **6**

8 Reproducible checks and limits of the evidence **7**

9 Technical decision **7**

1 Purpose, scope and correspondence with the code

The `horizon-range-v1` profile is in `labs/horizon/proof/src/lib.rs`. The demonstration effect is `record_claim_authorization`, in a local SQLite registry, not a payment. The intended economic use case is delegated insurance claim authorization; no solvent demand, automatic compliance or superiority over Vega is established.

The library verifies a *cryptographic statement*. The controller `packages/horizon-demo/src/store.ts` adds local expectations, nonce consumption, business deduplication and state management. The publication SDK `packages/proofkit-sdk` remains experimental: an interface does not replace any of these assumptions.

Object	Status of the argument
Algebraic identities and bounds	Mathematical derivations below.
Primitive and composition	Security conditional on explicit assumptions; inherited from cited work.
Code and local state	Adversarial tests and abstract finite model; no code refinement proof.
Octave/MATLAB	Exact numerical verification of examples and a reduced domain; Octave execution distinct from MATLAB.
SHA-256 manifests	Integrity relative to a trusted manifest; self-declared date, not a third-party timestamp, signature or proof of meaning.

2 Parameters and issuance contract

Let $\mathbb{G}$ be the Ristretto group of prime order

$$q = 2^{252} + 27742317777372353535851937790883648493.$$

Canonical encodings and the prime-order group are specified by RFC 9496 [4]. $G, H \in \mathbb{G}$ are the fixed bases from `PedersenGens::default()`. The prover does not choose them. Set $N = 2^{32}$.

Two authorities produce $A = \mathrm{Com}(a, r_a) = aG + r_aH$ and $C = \mathrm{Com}(c, r_c) = cG + r_cH$. The contract requires nonnegative integers representable as `u64`, the same EUR currency and the same exponent 2. Signatures cover profile, role, authority, tenant, subject, currency, exponent, version, holder key, period and commitment. Trust keys are pinned; the two authority identifiers and keys must be distinct.

Assumption 1 (Issuance and joining)**.** *Each authority computes or checks its commitment opening from its facts. It does not blindly sign a point submitted by the holder. The amount authority does not necessarily know the limit, and vice versa. Both openings reach the prover, which knows both values. Shared metadata actually refers to the same case and business version. The controller independently associates that subject with the target object.*

Two different keys do not prove organizational independence, quality of expertise or absence of collusion. Schema version 1 of the prototype is not an insurance claims history registry. A signature attests issuance under a key, not the truth of the signed fact.

3 Relation and conditional derivations

3.1 Commitment: confidentiality and binding

Lemma 2 (Hiding in the ideal model)**.** *For $H \neq 0$ and r uniform in $\mathbb{Z}_q$, the distribution of $vG + rH$ is uniform in $\mathbb{G}$ for any fixed value v.*

Proof. Multiplication $r \mapsto rH$ is a bijection because q is prime and $H \neq 0$. Translation by vG preserves uniformity. Commitment distributions are therefore equal for any two values. □

This result concerns the commitment alone. In the code, the quality of the random generator and conversion to scalars remains an implementation assumption. It does not justify masks that are reused or selected based on the secret.

Lemma 3 (Computational binding)**.** *Producing two openings* $(v, r) \neq (v', r')$ *of the same commitment with* $v \neq v' \pmod q$ *makes it possible to compute* $\log_G H$*.*

Proof. Equality gives $(v - v')G = (r' - r)H$. If $r' = r$, then $v = v' \pmod q$, a contradiction. Thus $(r' - r)^{-1}$ exists in $\mathbb{Z}_q$ and $\log_G H = (v - v')(r' - r)^{-1} \pmod q$. □

Confidentiality is information-theoretic in the ideal model; binding depends on the discrete logarithm and bases with no known relation [10, 3]. No reduced numerical experiment validates this assumption for Ristretto.

3.2 The statement actually proved

The derived commitments are

$$X = A - G, \qquad D = C - A.$$

The prover invokes an aggregated range proof for two 32-bit values [1, 2], with openings

$$x = a - 1, \quad d = c - a, \quad r_X = r_a, \quad r_D = r_c - r_a \pmod q.$$

The opening relation is

$$\exists x, d \in \{0, \ldots, N-1\},\ r_X, r_D \in \mathbb{Z}_q : X = xG + r_X H, \quad D = dG + r_D H. \tag{1}$$

A necessary distinction: since H generates the group, every point mathematically has an opening to any value. A mere existential membership statement is therefore insufficient to bind a business value. The argument uses the *argument of knowledge* of openings and computational binding to the openings actually attested.

Assumption 4 (Inherited primitive)**.** *The noninteractive Bulletproofs protocol used provides completeness, computational knowledge of openings and confidentiality appropriate to its security model and Fiat–Shamir transcripts. Its implementation, parameters, randomness and transcript analysis match that protocol. The corrected paper is the reference; this dossier does not redo its reduction or audit the library.*

The ePrint paper reports a correction to its Fiat–Shamir section in 2022 [1]. This history rules out treating a security citation as an automatic proof for every software adaptation.

Proposition 5 (Conditional completeness)**.** *If signatures and metadata are valid, the policy permits the mode, the challenge is the expected one and has not expired, and* $1 \leq a \leq N$*,* $a \leq c < a + N$*, then an honest prover constructs both derived commitments with admissible values. Under the primitive's completeness and in the absence of execution errors, its proof is accepted.*

Proof. $x = a - 1 \in [0, N-1]$ and $d = c - a \in [0, N-1]$. By linearity, $X = (a-1)G + r_a H$ and $D = (c-a)G + (r_c - r_a)H$. These are exactly the inputs expected by the verifier. It tests the other preconditions separately. □

This completeness guarantees neither network availability, absence of revocation after computation nor success of a business effect.

Proposition 6 (Integer bounds and absence of wraparound)**.** *Assume extraction of the openings in (1), binding of the commitments and issuance in the* `u64` *integer domain. Then the attested values satisfy*

$$1 \le a \le 2^{32}, \qquad 0 \le c - a < 2^{32}, \qquad c \le 2^{33} - 1.$$

Proof. The extracted openings give an opening of A to $a' = x + 1$ with mask r_X, and of C to $c' = x + 1 + d$ with mask $r_X + r_D$. They satisfy $1 \le a' \le N$ and $a' \le c' \le 2N - 1$. If a' or c' differ modulo q from the attested values, this violates binding. Otherwise $a = a' \pmod q$ and $c = c' \pmod q$. The attested values lie between 0 and $2^{64} - 1 < q$, and the extracted values between 0 and $2^{33} - 1 < q$. Two congruent integers in $[0, q)$ are equal; the bounds are therefore integer bounds, not merely modular ones. □

The limit may exceed $2^{32} - 1$: `u64` is needed. The maximum 8 589 934 591 cents equals 85 899 345.91 EUR, without any claim that this maximum suits an insurance contract.

Proposition 7 (Conditional computational soundness)**.** *Conditional on honest authorities for issuance, binding, the inherited argument of knowledge, signature unforgeability and a faithful implementation, accepting an attested value that violates the bounds in Proposition 6 requires breaking at least one of these assumptions or the context-binding checks.*

Proof. A signature on an unissued attestation is a forgery. For issued attestations, extraction and the previous proposition impose the bounds. An incorrect context association requires either an erroneous trusted input, an encoding/comparison error or a break in the mechanism that authenticates that context. □

A failure bound can be organized as a sum of forgery, collision, extraction-failure and binding-break events. Without a complete reduction and oracle model, this dossier supplies neither reduction constants nor a numerical security probability. The 500 rejections in a campaign are not an estimate of soundness.

4 Composition with signatures and challenge

The expected challenge is issued and stored by the controller. It contains tenant, client, audience, action, operation, object, subject, policy and epoch, trust epoch, 32-byte nonce, timestamps and method. Equality covers *all* these fields before authorization. The agent does not supply trust roots. Encodings use typed JSON in declaration order with separate domains; this is not a promise of universal canonical JSON.

Proposition 8 (Presentation binding)**.** *Assume Ed25519 is unforgeable for honest keys, SHA-256 is collision-resistant, domains are distinct and encoding is injective over the admitted types. A presentation accepted under an independent challenge is bound to the attestation pair, payload and challenge by the holder key.*

Proof. The holder signature covers the digest of the profile, attestations, challenge and payload tuple, itself encoded under a separate domain. Changing any of these objects without obtaining a new signature requires a collision or forgery. An input with another challenge is rejected by equality with the expected value. Authority signatures cover each attestation body separately. □

RFC 8032 describes EdDSA [5]; the code uses strict verification and rejects weak keys. A compromised key or malicious authority lies outside the assumption, and two identical presentations can still be replayed without consumption state.

Mode A. The entire challenge is also added to the Merlin transcript. The range proof is recomputed for each presentation. This is cryptographic context binding, not a clock or proof of instantaneous source truth.

Mode B. The range proof is prepared over immutable attestations and retained. A new holder signature covers the challenge and the exact proof. This attests fresh authorization by that key over a historical proof, *not* fresh knowledge of the witness. The current verifier checks the entire proof on every call.

Policy C. If confidentiality is not required, the selector chooses signed disclosure. Otherwise, it chooses B if reuse is permitted and A if not. Cryptographic rejection never triggers a fallback. Method authorization remains independently verified.

5 Replay prevention and transaction boundary

The controller's operation identity is the hash of the tenant, audience, action and object tuple. It does not contain the nonce. The guarantee is local to this deduplication domain; multiple audiences or independent databases do not automatically share a global business identity.

Assumption 9 (Storage and target model)**.** *Transactions are serialized and atomic. Committed states persist without backup rollback, administrative deletion or corruption. All effect paths pass through the same controller, the same stable business identity and the same database. The* `Verified` *result comes from the trusted cryptographic worker, never from a forged HTTP field. The clock and trust roots are controlled by the verifier.*

Proposition 10 (At most one local effect per business identity)**.** *Under the preceding assumption, the controller produces at most one effect row for an operation identity, including with multiple nonces, concurrency, a crash after reservation or an ambiguous response.*

Proof. Initially, no operation exists. A transaction validates the current context, consumes the nonce and inserts the operation under a unique primary key. Any subsequent reservation with that key is rejected. The target transaction requires the reserved state, rechecks deadlines and epochs, then inserts the effect under the same primary key and completes the operation. Failure of this recheck preserves a rejected operation; an ambiguous response preserves an unknown state. None of these states is deleted to retry. Reconciliation queries existing effects and does not redispatch. By induction over transitions, the number of effects remains at most one. □

SQLite provides transaction mechanisms under its documented assumptions [12, 13]. The proof is an abstract invariant, not a theorem verified over the engine, filesystem or TypeScript code. It proves neither that an effect will eventually occur nor atomicity over a remote API. A crash after reservation may leave an operation blocked. The chained log detects certain modifications if a root is known, but an administrator able to rewrite the entire database is not excluded by a mere local hash.

Temporal validity is the intersection of intervals:

$$t_{\min} = \max(t_{\text{challenge}}, t_A, t_C), \qquad t_{\max} = \min(e_{\text{challenge}}, e_A, e_C), \qquad t_{\min} \le t < t_{\max}.$$

Revocations and epochs are rechecked after the cryptographic wait and before the effect. This bounds a snapshot's validity without proving the freshness of an external system unavailable during a crisis.

6 Leakage and business counterexamples

Values and openings are private from the verifier in A/B under the proof's assumptions. Authorities know their own values, the prover knows both, and disclosed mode reveals them all. Commitments, signatures, keys, opaque case identifier, tenant, roles, timestamps, policy and decision are public. Fixed commitments make sessions linkable; rerandomizing the proof alone

does not make them anonymous. The decision reveals at least the inequality and bounds. Adaptive policies, auxiliary information and timing channels may reveal more; no system-level noninterference is demonstrated.

Counterexample	Accepted statement	What is missing
Cumulative budget	Two cases of 60 each satisfy $60 \leq 100$.	Their sum 120 exceeds a shared budget of 100; a concurrent ledger is needed.
Deductible	A payment of 15000 is below 20000.	With a loss of 15000 and a deductible of 5000, the expected amount would be 10000; this formula is not proved.
Exponent	Integers $1000 \leq 10000$ pass.	1000 EUR exceeds 10000 cents; the prototype prevents this mismatch by fixing EUR/exponent 2.
Currency	Two equal numbers.	EUR and USD are not a common unit; conversion, source and time must be authenticated.
Wraparound	$103 \equiv 2 \pmod{101}$.	An unbounded source domain may change the integer meaning; the `u64` contract excludes this case for the real q.
Nonce alone	Two different nonces are each fresh.	They may authorize the same object twice without business reservation.

These examples do not bypass existing controls; they delimit the statement. Complete insurance claim logic must specify exclusions, deductibles, taxes, coinsurance, coverage dates and reserves. No circuit becomes "perfect" merely by increasing its speed.

7 CPU, GPU and Vega measurements, and quantum assumptions

Measurements retained in `docs/horizon/BENCHMARK_RESULTS.json` use 500 observations after 30 warmups. Linux cryptographic functions cover five cases; quantiles use the upper rank. IID independence of observations is not established, and confidence intervals are conditional on it. The same fixed public parameters are initialized once per process with `OnceLock`: no proof, witness, verdict or randomness is cached there. Mode B retains its prepared proof separately, in accordance with its semantics.

Published laboratory measurement	Observation, not an SLA
A, CPU generation p95 before / after	7.770 / 6.546 ms
B, full CPU verification p95 before / after	2.266 / 1.247 ms
B, signed presentation p95 after preparation	0.065 ms; not a new proof
Aggregated proof alone	672 bytes; envelope and signatures excluded
Disclosed baseline, verification p95	0.245 ms
Windows IPC, verification and durable local effect	p95 13.014 ms; maximum 180.310 ms, excluding HTTP
Vega MC SHA-256 2 KiB, 8 threads	Proof 65.836 ms, verification 19.885 ms; Criterion central estimates, not p95

The proof size here is $32(2\log_2(2 \cdot 32) + 9) = 672$ bytes for the aggregated format used. It does not include public commitments or authorizations. The observed p95 reduction from retaining parameters is 15.75% for A and 44.97% for B verification; no external record follows.

Vega addresses credential presentations and reusable preparation [7, 6]. The public repository's SHA-256 benchmark is not the full application circuit behind the paper's 92/23 ms figures. Horizon

instead requires adapted issuance over commitments and leaves signatures and metadata public. Relations, leakage models and phases differ: no speed ratio supports concluding that one beats the other on a shared task.

The GPU dossier is retained separately in `docs/experiments/gpu/`. A CUDA/RISC Zero attempt is not a GPU port of this Bulletproofs relation. This text attributes no GPU speedup without a reproduced result, a statement of the relation, size, host/device transfer, preparation, independent verification and exact hardware. CUDA unavailability is a negative feasibility result, not a speed measurement. GPU figures are therefore not interpolated from CPU figures.

Finally, GPU acceleration and quantum resistance are different axes. Efficient discrete-logarithm computation would compromise binding and elliptic curve signatures; Shor's algorithm motivates this distinction [11]. ML-DSA and SLH-DSA are post-quantum signature standards [8, 9]. Replacing Ed25519 with one of them would not make Pedersen/Bulletproofs post-quantum. Using SHA-256 or SHA-3 in a transcript does not turn an elliptic curve protocol into an entirely hash-based proof. Migration requires another proof relation, a composition analysis and new benchmarks.

8 Reproducible checks and limits of the evidence

`math/horizon_checks.m` enumerates integer pairs in a reduced domain for 1 to 6 bits, including negative values; checks 32-bit boundaries using `uint64`; reconstructs q through decimal strings; and explores all reachable states of a finite reservation model with two nonces and one operation. Decimal arithmetic functions handle only small digits, never q in floating point. No curve, signature or Bulletproofs implementation is recreated in MATLAB/Octave.

The JSON result explicitly states runtime, version, UTC date and limitations. The container pins the base digest, and the dossier retains the built image identifier and package versions. The Dockerfile still depends on the package repository at rebuild time: this is not a promise of bit-for-bit reproducible builds. Execution has no network after building. MATLAB requires a separate runtime/license: an Octave success is not presented as a MATLAB execution.

Existing Rust and Node adversarial tests remain separate experimental evidence. The reduced model replaces neither verification of a real cryptographic proof by an independent process, a model checker applied to the code nor an independent security review. The PDF compilation log and page inspection attest only to the document's production.

9 Technical decision

The defensible result is a feasibility laboratory: a narrow relation, explicit assumptions, real proofs and a local controller that rejects before its effect. Production security still requires qualified authorities, key management, revocation, isolation, audit, a remote recovery model and validation of the economic need. Cloud deployments remain network/IAM configuration examples, not operational certification. No theorem presented here implies compliance, absence of competition or universally correct business logic.

References

[1] Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. Cryptology ePrint Archive, Report 2017/1066, 2017. URL https://eprint.iacr.org/2017/1066. Published IEEE S&P 2018; revision 2022-04-14 corrects a Fiat–Shamir-section mistake. Consulted 2026-10-17.

[2] Dalek Cryptography. Bulletproofs 5.0.0: RangeProof API, 2026. URL https://docs.rs/bulletproofs/5.0.0/bulletproofs/struct.RangeProof.html. Version-pinned implementation documentation; consulted 2026-10-17. Year denotes consultation, not release.

[3] Dalek Cryptography. Bulletproofs implementation notes, 2026. URL https://doc-internal.dalek.rs/bulletproofs/notes/index.html. Primary implementation explanation, not an audit of Horizon. Consulted 2026-10-17.

[4] Henry de Valence, Jack Grigg, Mike Hamburg, Isis Lovecruft, George Tankersley, and Filippo Valsorda. The ristretto255 and decaf448 groups. RFC 9496, Informational, 2023. URL https://www.rfc-editor.org/rfc/rfc9496.html.

[5] Simon Josefsson and Ilari Liusvaara. Edwards-curve digital signature algorithm (EdDSA). RFC 8032, Informational, 2017. URL https://www.rfc-editor.org/rfc/rfc8032.html.

[6] Microsoft. vega-prover, 2026. URL https://github.com/microsoft/vega-prover/tree/c0ee259053cd12eaf43ed71b5cde375452b3ee4d. Exact source commit used for repository SHA-256 benches. Not the full credential benchmark.

[7] Microsoft Research. Vega: Low-latency zero-knowledge proofs over existing credentials, 2026. URL https://www.microsoft.com/en-us/research/wp-content/uploads/2026/05/Vega_Low-Latency-Zero-Knowledge-Proofs-over-Existing-Credentials.pdf. Primary paper, May 2026 PDF; section 8. Consulted 2026-10-17.

[8] National Institute of Standards and Technology. Module-lattice-based digital signature standard. FIPS 204, 2024. URL https://csrc.nist.gov/pubs/fips/204/final. Consulted 2026-10-17; the landing page lists potential errata dated 2026-07-31.

[9] National Institute of Standards and Technology. Stateless hash-based digital signature standard. FIPS 205, 2024. URL https://csrc.nist.gov/pubs/fips/205/final. Consulted 2026-10-17.

[10] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Advances in Cryptology – CRYPTO '91*, volume 576 of *Lecture Notes in Computer Science*, pages 129–140, 1992. doi: 10.1007/3-540-46766-1_9. URL https://doi.org/10.1007/3-540-46766-1_9. Conference 1991; published volume 1992. Publisher metadata consulted 2026-10-17.

[11] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer, 1996. URL https://arxiv.org/abs/quant-ph/9508027v2. Expanded version of the 1994 FOCS paper.

[12] SQLite. Transaction, 2026. URL https://www.sqlite.org/lang_transaction.html. Consulted 2026-10-17.

[13] SQLite. Atomic commit in SQLite, 2026. URL https://www.sqlite.org/atomiccommit.html. Filesystem and hardware assumptions matter; consulted 2026-10-17.